Практическая работа №3. Параметризованные SQL-запросы и SqlParameter

Цель работы

Научиться передавать данные из C#-приложения в SQL-запросы с использованием параметров.

В ходе работы необходимо:

- создать параметризованные SQL-запросы;
- использовать `SqlParameter`;
- выполнять параметризованные `SELECT`, `INSERT`, `UPDATE` и `DELETE`;
- понять отличие параметризованного запроса от конкатенации строк;
- закрепить принцип защиты от SQL-инъекций.

Задание 1. Подготовка проекта

Используйте проект из предыдущей практической работы или создайте новый:

```
dotnet new console -n AdoNetPractice03
```

Перейдите в каталог проекта:

```
cd AdoNetPractice03
```

Установите пакет:

```
dotnet add package Microsoft.Data.SqlClient
```

Добавьте:

```
using Microsoft.Data.SqlClient;
```

Создайте подключение:

```
string connectionString =
    "Server=localhost;Database=coffee_shop;Trusted_Connection
=True;TrustServerCertificate=True;";

using var connection = new SqlConnection(connectionString);

connection.Open();

Console.WriteLine("Подключение установлено.");
```

Задание 2. Поиск товара по ID

Начнём с простой операции.

Пользователь должен указать ID товара:

```
Console.Write("Введите ID товара: ");

int productId = Convert.ToInt32(Console.ReadLine());
```

Создайте параметризованный запрос:

```
SELECT id, name, price
FROM products
WHERE id = @id;
```

Создайте команду:

```
string sql = """
    SELECT id, name, price
    FROM products
    WHERE id = @id;
    """;

using var command = new SqlCommand(sql, connection);
```

Добавьте параметр:

```
command.Parameters.AddWithValue("@id", productId);
```

После этого выполните запрос:

```
using var reader = command.ExecuteReader();

while (reader.Read())
{
    Console.WriteLine(
        $"{reader["id"]}: {reader["name"]} — {reader["pric
e"]} руб."
    );
}
```

Проверьте работу программы с разными ID.

Например:

```
1
3
10
16
```

Задание 3. Поиск товара по названию

Теперь пользователь должен иметь возможность искать товар по названию.

Получите название:

```
Console.Write("Введите название товара: ");

string name = Console.ReadLine()!;
```

Используйте запрос:

```
SELECT id, name, price
FROM products
WHERE name = @name;
```

Добавьте параметр:

```
command.Parameters.AddWithValue("@name", name);
```

Для выполнения используйте `ExecuteReader()`.

Проверьте работу, например, с:

```
Капучино
Латте
Чизкейк
Круассан
```

Задание 4. Добавление товара

Теперь необходимо реализовать добавление нового товара.

Получите от пользователя:

- название;
- ID категории;
- цену.

Например:

```
Console.Write("Название товара: ");
string name = Console.ReadLine()!;

Console.Write("ID категории: ");
int categoryId = Convert.ToInt32(Console.ReadLine());

Console.Write("Цена: ");
decimal price = Convert.ToDecimal(Console.ReadLine());
```

Используйте параметризованный запрос:

```
INSERT INTO products (name, category_id, price)
VALUES (@name, @category_id, @price);
```

Добавьте параметры:

```
command.Parameters.AddWithValue("@name", name);
command.Parameters.AddWithValue("@category_id", categoryId);
command.Parameters.AddWithValue("@price", price);
```

Выполните:

```
int rowsAffected = command.ExecuteNonQuery();

Console.WriteLine($"Добавлено строк: {rowsAffected}");
```

Проверьте результат в SQL Server.

Задание 5. Изменение цены товара

Теперь реализуйте изменение цены существующего товара.

Пользователь вводит:

```
ID товара
```

```
Новая цена
```

SQL:

```
UPDATE products
SET price = @price
WHERE id = @id;
```

C#:

```
command.Parameters.AddWithValue("@price", price);
command.Parameters.AddWithValue("@id", productId);
```

Выполните запрос через:

```
ExecuteNonQuery()
```

После выполнения выведите:

```
Изменено строк: 1
```

Если товар с таким ID отсутствует:

```
Изменено строк: 0
```

Обратите внимание на разницу.

Задание 6. Удаление товара

Реализуйте удаление товара по ID.

SQL:

```
DELETE FROM products
```

```
WHERE id = @id;
```

Параметр:

```
command.Parameters.AddWithValue("@id", productId);
```

Выполните:

```
int rowsAffected = command.ExecuteNonQuery();
```

Выведите количество удалённых строк.

После этого проверьте результат в SQL Server.

Задание 7. Получение статистики через параметры

Теперь используем параметр вместе с `ExecuteScalar()`.

Пользователь вводит минимальную цену:

```
Console.Write("Введите минимальную цену: ");

decimal minPrice =
    Convert.ToDecimal(Console.ReadLine());
```

Необходимо определить количество товаров дороже указанной цены.

SQL:

```
SELECT COUNT(*)
FROM products
WHERE price >= @min_price;
```

Добавьте параметр:

```
command.Parameters.AddWithValue("@min_price", minPrice);
```

Получите результат:

```
int count =
    Convert.ToInt32(command.ExecuteScalar());

Console.WriteLine(
    $"Товаров с ценой от {minPrice}: {count}"
);
```

Задание 8. Поиск клиентов по имени

Перейдите к таблице `customers`.

Пользователь вводит имя или фамилию:

```
Console.Write("Введите имя или фамилию: ");

string search = Console.ReadLine()!;
```

Используйте параметризованный запрос:

```
SELECT id, full_name, phone, email
FROM customers
WHERE full_name LIKE @search;
```

В качестве параметра необходимо передать:

```
command.Parameters.AddWithValue(
    "@search",
    $"%{search}%"
);
```

Получите результаты через `ExecuteReader()`.

Например, при вводе:

```
Иван
```

должен найтись:

```
Иван Петров
```

Задание 9. Сравнение двух подходов

Теперь необходимо посмотреть на проблему с другой стороны.

Ниже приведён небезопасный вариант:

```
string sql =
    $"SELECT id, name, price " +
    $"FROM products " +
    $"WHERE name = '{name}'";
```

Обратите внимание: значение переменной `name` непосредственно вставляется в SQL-строку.

Теперь создайте безопасный вариант:

```
string sql = """
    SELECT id, name, price
    FROM products
    WHERE name = @name;
    """;

using var command = new SqlCommand(sql, connection);

command.Parameters.AddWithValue("@name", name);
```

Сравните оба варианта.

Ответьте письменно:

1. Где находится пользовательское значение в первом варианте?
2. Где находится пользовательское значение во втором варианте?
3. Почему параметризованный вариант безопаснее?
4. Что происходит с SQL-кодом и данными в каждом случае?

Самостоятельная часть

Теперь самостоятельно реализуйте небольшой модуль поиска и управления товарами.

Программа должна предоставить пользователю следующие операции:

```
=== УПРАВЛЕНИЕ ТОВАРАМИ ===

1. Найти товар по ID
2. Найти товар по названию
3. Добавить товар
4. Изменить цену
5. Удалить товар
6. Найти товары дороже указанной цены
0. Выход
```

Для выбора операции используйте:

```
Console.ReadLine();
```

и:

```
switch
```

Каждая операция должна использовать параметризованный SQL-запрос.

Требования к самостоятельной части

Поиск по ID

Использовать:

```
SqlCommand
SqlParameter
ExecuteReader()
```

Поиск по названию

Использовать:

```
SqlCommand
SqlParameter
ExecuteReader()
```

Добавление товара

Использовать:

```
SqlCommand
SqlParameter
ExecuteNonQuery()
```

Изменение цены

Использовать:

```
SqlCommand
SqlParameter
ExecuteNonQuery()
```

Удаление товара

Использовать:

```
SqlCommand
SqlParameter
```

```
ExecuteNonQuery()
```

Поиск товаров по цене

Использовать:

```
SqlCommand
SqlParameter
ExecuteReader()
```

или `ExecuteScalar()`, если вы считаете количество найденных товаров.

Дополнительное задание

Переделайте добавление товара так, чтобы параметры создавались с явным указанием типа.

Например:

```
command.Parameters.Add(
    "@name",
    SqlDbType.NVarChar,
    150
).Value = name;
```

Для категории:

```
command.Parameters.Add(
    "@category_id",
    SqlDbType.Int
).Value = categoryId;
```

Для цены:

```
command.Parameters.Add(
    "@price",
    SqlDbType.Decimal
).Value = price;
```

Сравните этот вариант с:

```
command.Parameters.AddWithValue(
    "@price",
    price
);
```

Что необходимо продемонстрировать преподавателю

Готовая работа должна содержать:

1. Параметризованный `SELECT`.
2. Параметризованный `INSERT`.
3. Параметризованный `UPDATE`.
4. Параметризованный `DELETE`.
5. Использование `SqlParameter`.
6. Использование `ExecuteReader()`.
7. Использование `ExecuteScalar()` или параметризованного запроса с ним.
8. Использование `ExecuteNonQuery()`.
9. Работу с пользовательским вводом.
10. Отсутствие конкатенации пользовательских данных с SQL-строкой в основных операциях.

Выводы по работе

В ходе практической работы были созданы параметризованные SQL-запросы для работы с базой данных `coffee_shop`.

Было изучено использование `SqlParameter` при выполнении операций:

```
SELECT
INSERT
UPDATE
DELETE
```

Параметризованные запросы позволяют отделить SQL-код от передаваемых данных и являются основным способом защиты SQL-запросов от SQL-инъекций.

В дальнейших практических работах параметры будут использоваться как стандартный способ передачи значений из C#-приложения в SQL Server.